



COM 682

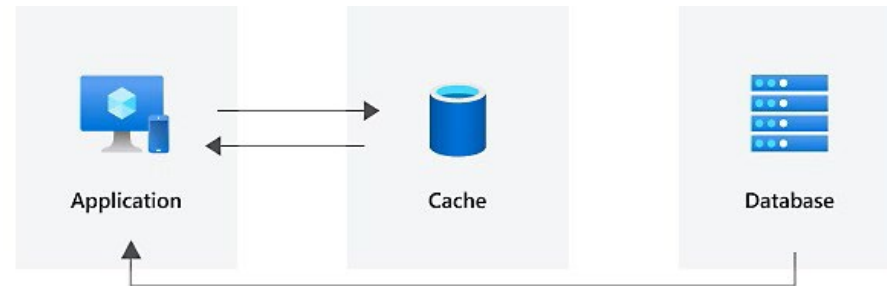
Cloud Native Development

Catching, Content Delivery Network, and Scalability

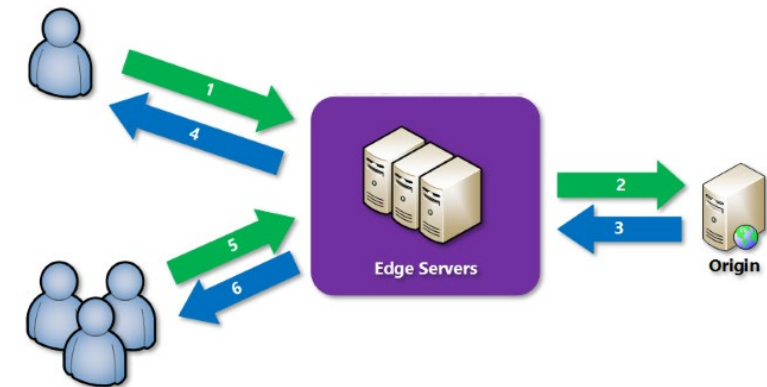
Zeeshan Tariq z.tariq@ulster.ac.uk

Agenda

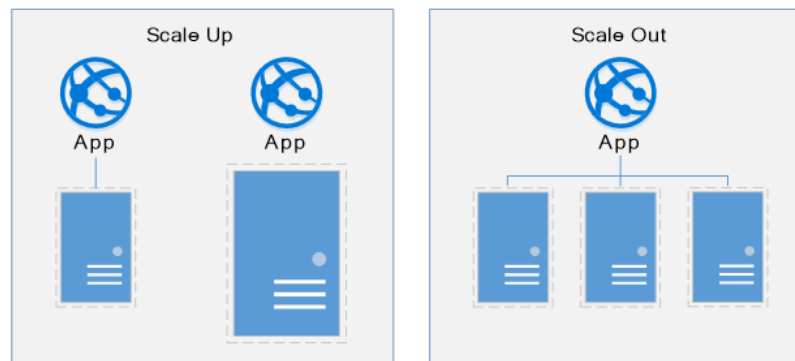
Caching



Content Delivery Network



Scalability



Caching

Overview

- Cache is a hardware or software component that stores data so that future requests for that data can be served faster;
- The data stored in a cache might be the result of an earlier computation or a copy of data stored elsewhere.
- A cache hit occurs when the requested data can be found in a cache, while a cache miss occurs when it cannot.
- Cache hits are served by reading data from the cache, which is faster than recomputing a result or reading from a slower data store; thus, the more requests that can be served from the cache, the faster the system performs.

Caching

Overview

- Developers and IT professionals use caching to save and **access key-value data in temporary memory faster and with less effort** than data stored in conventional data storage.
- Caches are useful in multiple scenarios with multiple technologies, such as:
 - Computer **caching with random access memory (RAM)**
 - Network caching on a **content delivery network**
 - A **web cache for web multimedia data**
- You can use caching to reduce database costs, deliver higher throughput and lower latency than most databases can offer, and **boost the performance of cloud and web applications.**

Types of Caching

Your caching strategy depends on how your application reads and writes data.

Is your application write-heavy, or is data written once and read frequently?

Is the data that's returned always unique?

Different data access patterns will influence how you configure a cache.

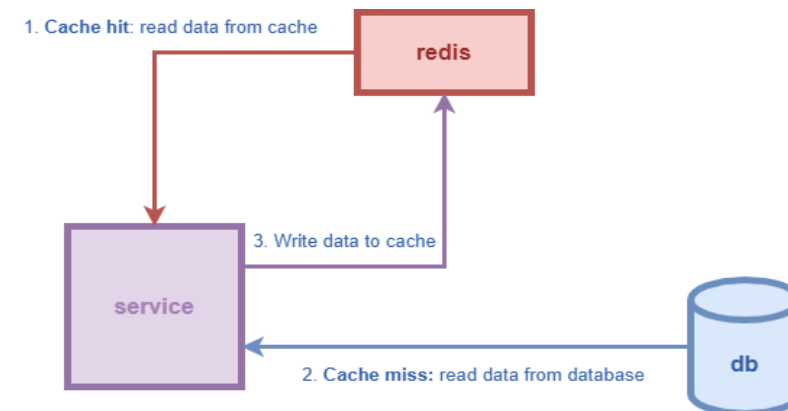
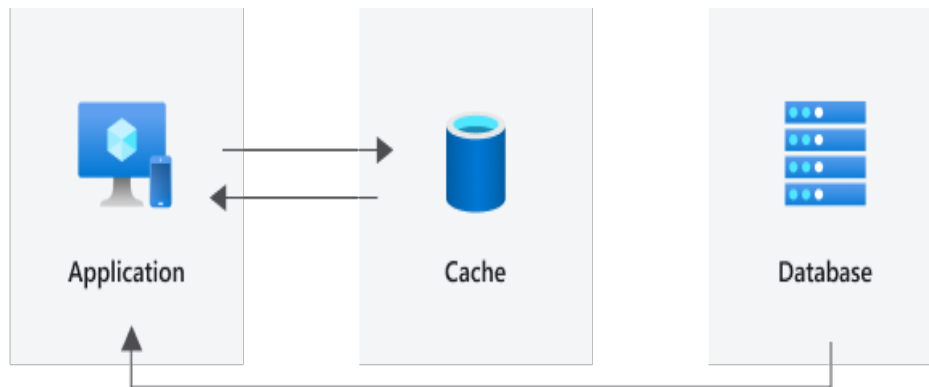
Common caching types include:

- **Cache-aside**
- **Read-through/write-through**
- **Write-behind/write-back**

Types of Caching

Cache-aside

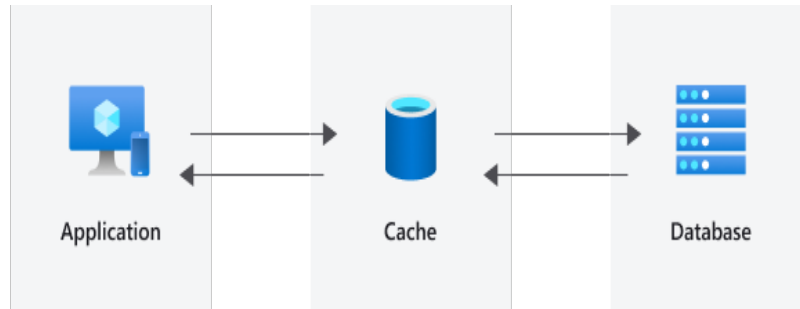
- For applications with read-heavy workloads, developers often use a cache-aside programming pattern, or "side-cache."
- They locate the side-cache outside the application, which can then connect with the cache to query and retrieve data or directly with the database **if the data isn't in the cache**. As the application retrieves the data, it copies it to the cache for future queries.
- You can use a side-cache to help improve application performance, maintain consistency between the cache and the data store, and keep data in the cache from getting stale.



Types of Caching

Read-through/write-through cache

- Read-through caches keep themselves up to date, while with write-through caching, **the application writes data to the cache and then to the database**. Both caches sit in line with the database, and the application treats them as the main data store.
- **Read-through caches help simplify applications where the same data is requested over and over, but the cache itself is more complex**, while the two-step write-through process can create latency.
- With read-through/write-through caching, developers can simplify application code, increase cache scalability, and minimize database load.



Read-Through Cache



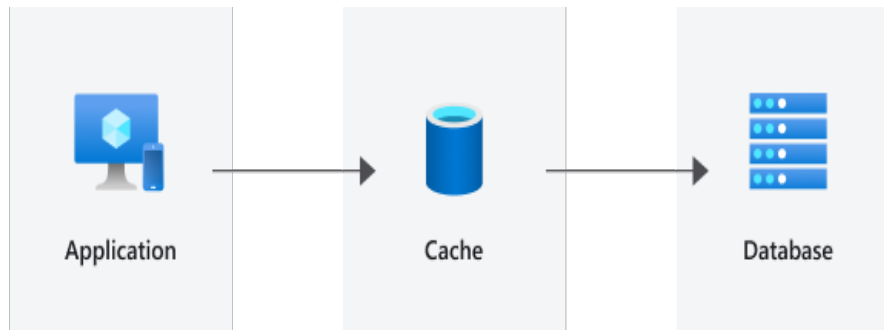
Write-Through Cache



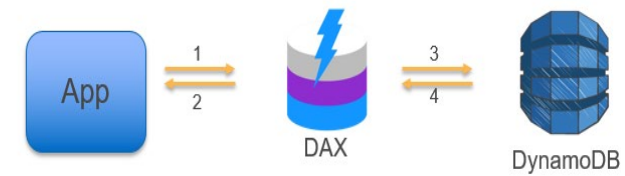
Types of Caching

Write-behind/write-back cache

- In this scenario, the **application writes data to the cache**, which is immediately acknowledged, and **then the cache itself writes the data back to the database** in the background.
- Write-behind caches, sometimes known as write-back caches, are best for write-heavy workloads, and **they improve write performance because the application doesn't need to wait for the write to complete before moving to the next task.**



Write-Back Cache



What is caching used for?

Output caching

Output caching helps increase webpage performance by storing the full source code of pages such as HTML and client scripts that a server sends to browsers for rendering.

Every time a user views the page, the server caches the output code in the application's memory. This enables the application to serve requests without running page code or communicating with other servers.

Data caching and database caching

Database speed and throughput can be key factors in overall application performance.

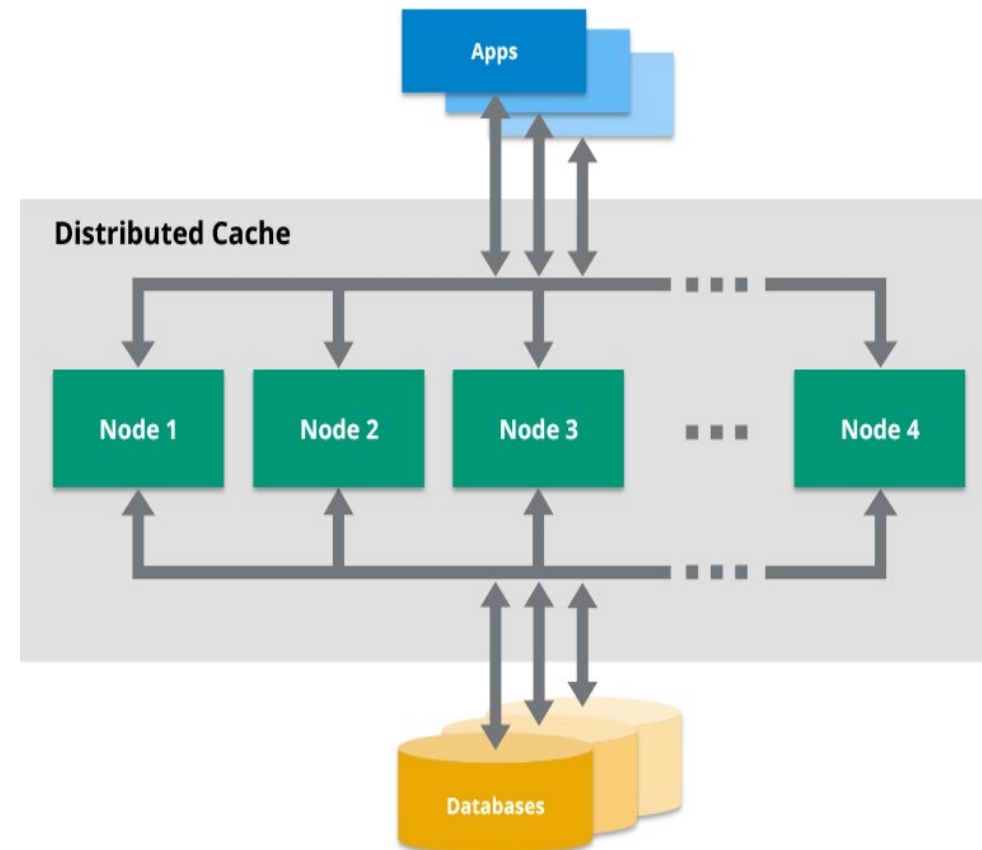
Database caching is used for frequent calls to data that doesn't change often, such as pricing or inventory data. It helps websites and applications load faster while increasing throughput and lowering data retrieval latency from back-end databases.

Caching

- Caching in Azure is an in-memory data store, which is a **distributed caching feature**.

A distributed cache is **an extension of the traditional concept of cache used in a single locale**.

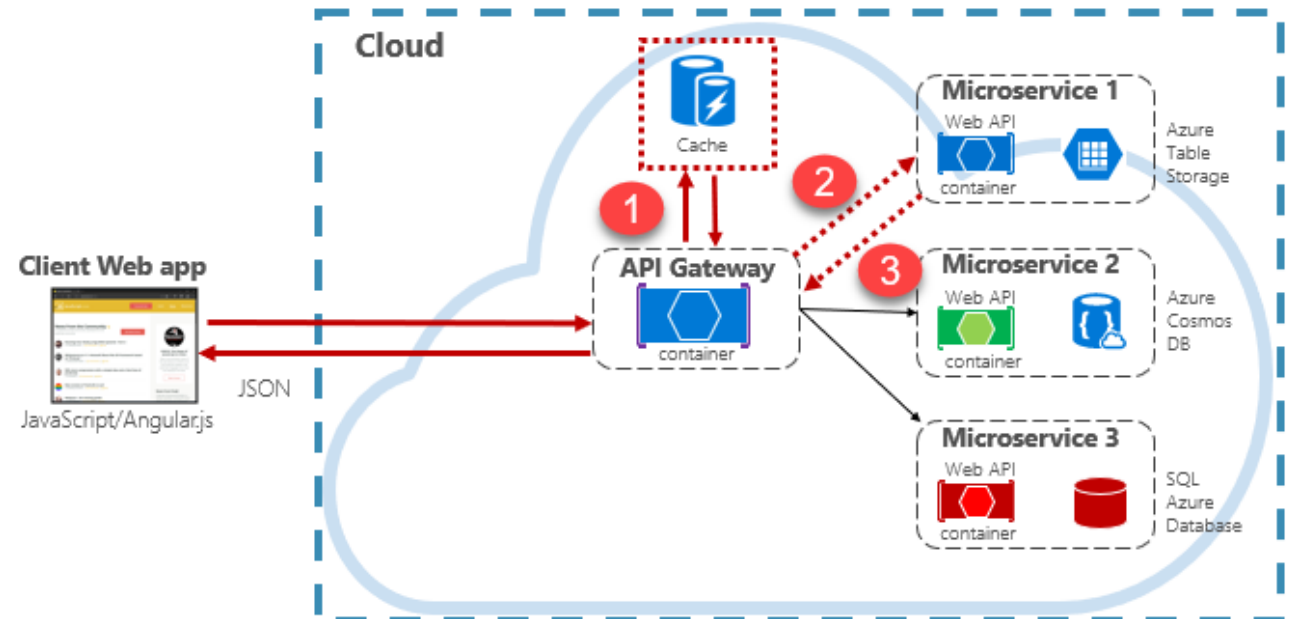
A **distributed cache may span multiple servers so that it can grow in size and in transactional capacity**. It is mainly used to store application data residing in the database and web session data.



Caching

The benefits of caching are well understood. The technique works by temporarily copying frequently accessed data from a backend data store to *fast storage* that's located closer to the application. Caching is often implemented where...

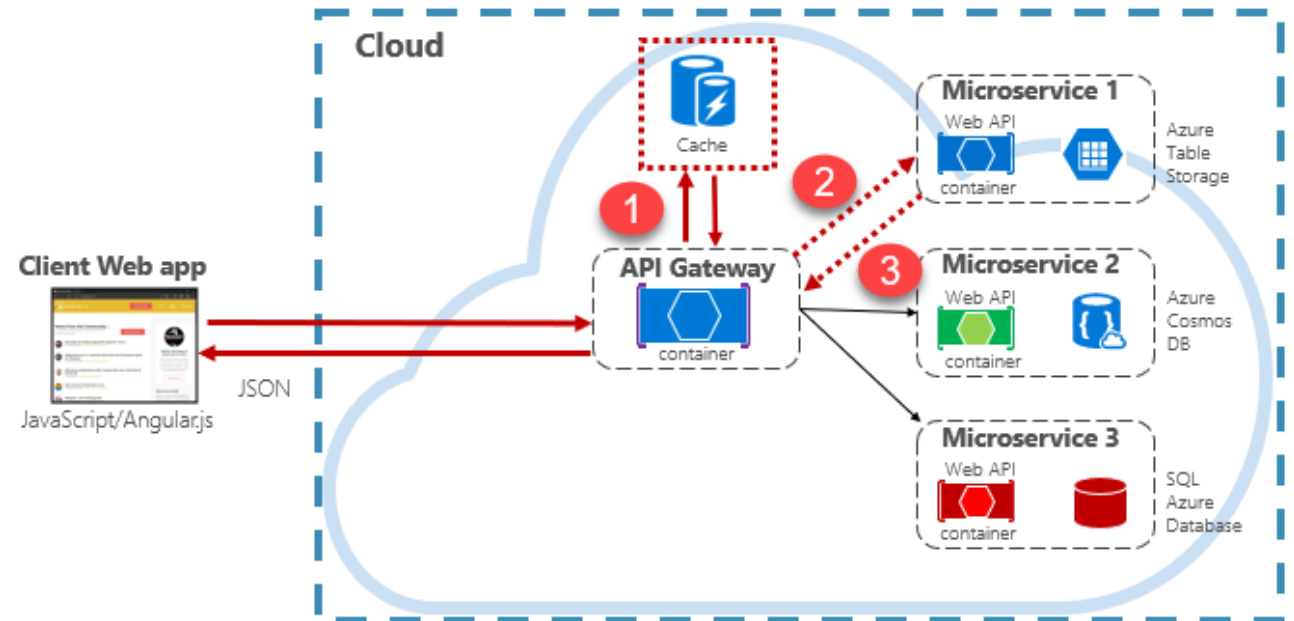
- Data remains relatively static.
- Data access is slow, especially compared to the speed of the cache.
- Data is subject to high levels of contention.



Caching in a cloud-native app

The benefits of caching are well understood. The technique works by temporarily copying frequently accessed data from a backend data store to *fast storage* that's located closer to the application. Caching is often implemented where...

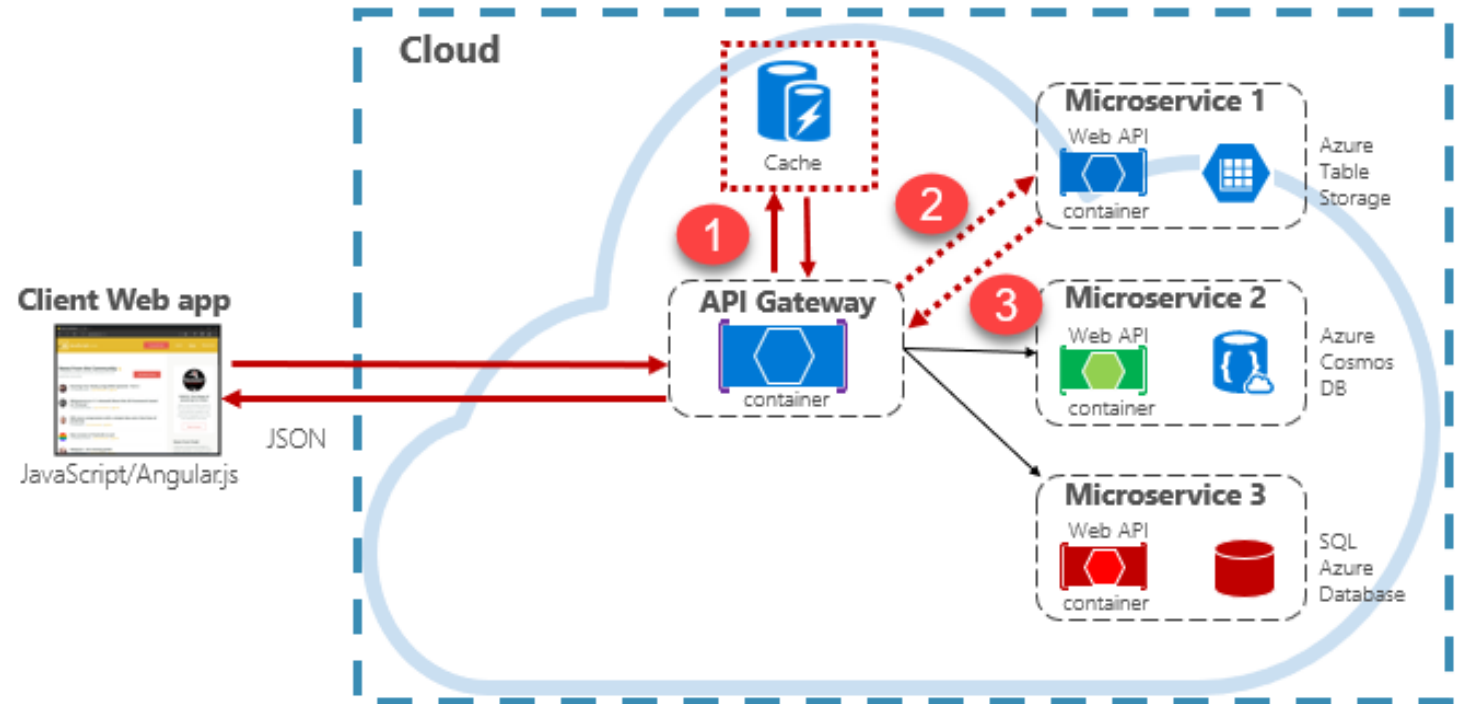
- Data remains relatively static.
- Data access is slow, especially compared to the speed of the cache.
- Data is subject to high levels of contention.



Caching in a cloud-native app

Cloud native applications typically implement a distributed caching architecture. The cache is hosted as a cloud-based backing service, separate from the microservices.

The distributed cache increases system responsiveness by returning cached data whenever possible. Additionally, separating the cache from the services allows the cache to scale up or out independently to meet increased traffic demands



Cache use case-1

E-commerce platform

 Walmart	▼	 Amazon	▼	 eBay	▼
 Etsy	▼	 Shopify	▼	 AliExpress	▼
 Ozon	▼	 BigCommerce	▼	 Flipkart	▼

•**Problem:** A high-traffic e-commerce site is slowed down by frequent database queries for product details.

•**Solution:** Use Azure Cache for Redis to cache product catalog data, such as product names, descriptions, and pricing.

•**Benefit:** When a user views a product, the application first checks the cache. If the data is present, it's returned instantly. This significantly reduces the load on the backend database and makes the website load much faster.

Cache use case-2

Financial trading platform



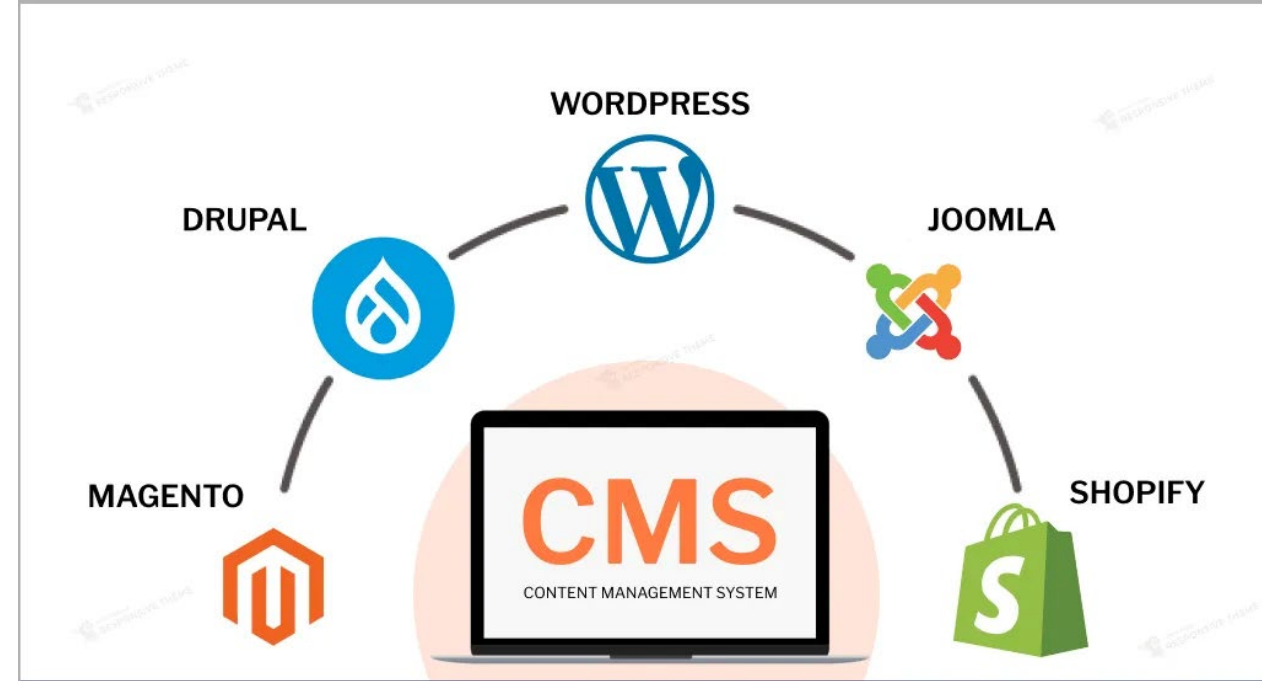
•**Problem:** Traders need to see stock quotes with minimal delay to make timely decisions.

•**Solution:** Cache real-time stock quotes in Azure Cache for Redis.

•**Benefit:** The platform can deliver low-latency, real-time updates to users because the most recent data is stored in a fast, in-memory cache

Cache use case-3

Content management system (CMS)



•**Problem:** A CMS is generating the same web pages repeatedly, which puts a strain on the database.

•**Solution:** Cache frequently requested pages and content fragments in Azure Cache for Redis.

•**Benefit:** This improves website performance by reducing the need to regenerate content and query the database for every request.

Other Cache solutions:

Azure Cache for Redis: A managed service that provides a high-performance, scalable, and secure Redis cache. It is the most common and recommended solution for general-purpose caching needs in Azure.

Database-as-a-cache: You can use other Azure databases like [Azure SQL Database](#) or [Azure Cosmos DB](#) as a cache with the cache-aside pattern. This involves implementing caching logic within your application to check the cache first before querying the database.

Custom solutions: Depending on your specific needs, you might also build custom caching solutions using other Azure services like [Azure Functions](#) or [Azure App Service](#).

Azure CDN: Use Content Delivery Network for caching static content at the edge of the network, closer to users, to improve latency and performance for global audiences.

Microsoft Orleans In-Memory Data Grid: For .NET applications, Orleans provides an in-memory data grid that can be used for caching without requiring an external service like Redis.



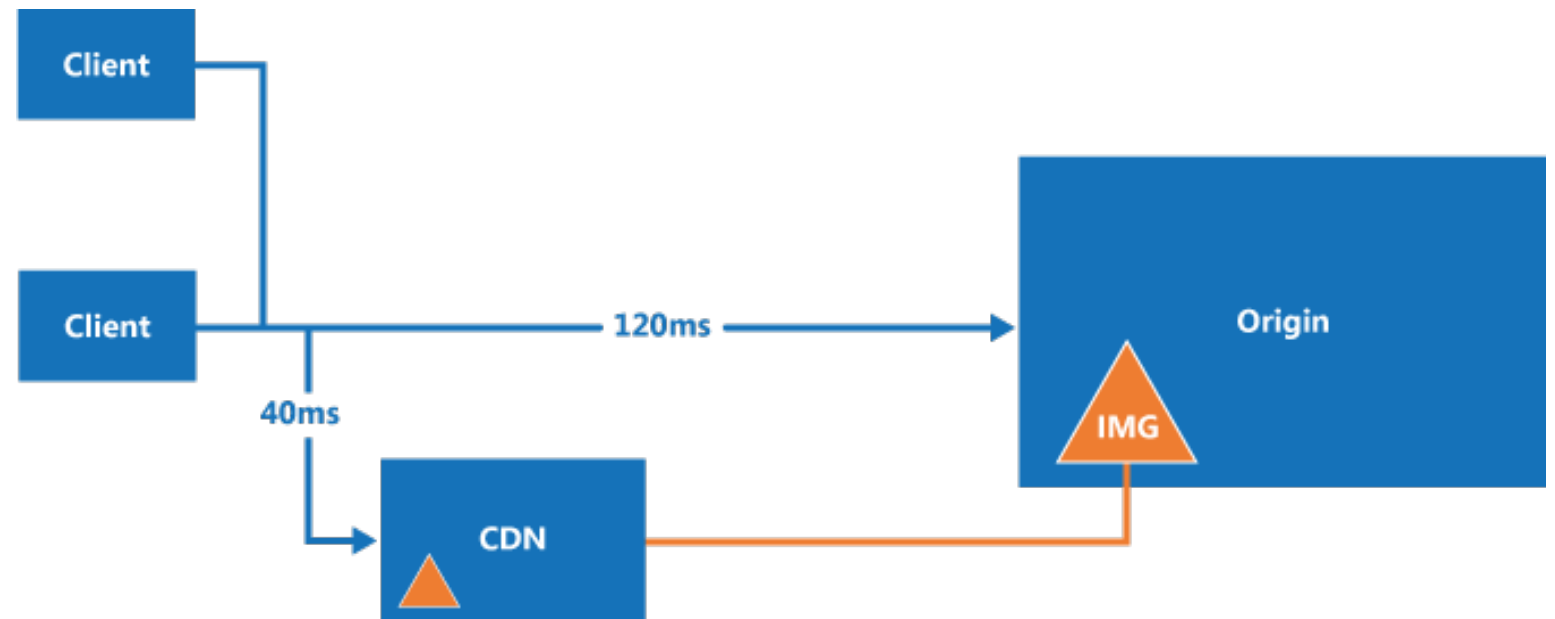
Content Delivery Network



Content Delivery Network (CDN)

A content delivery network (CDN) is a distributed network of servers that can efficiently deliver web content to users. **CDNs' store cached content on edge servers in point-of-presence (POP) locations** that are close to end users, to minimize latency.

Azure Content Delivery Network (CDN) offers developers a global solution for rapidly delivering high-bandwidth content to users by caching their content at strategically placed physical nodes across the world.



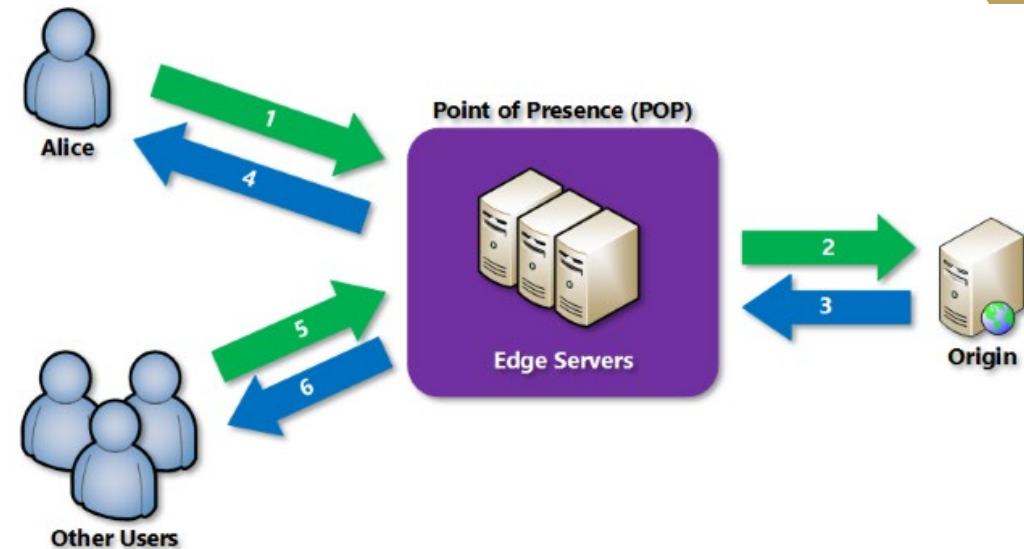
Content Delivery Network (CDN)

Microsoft Regions



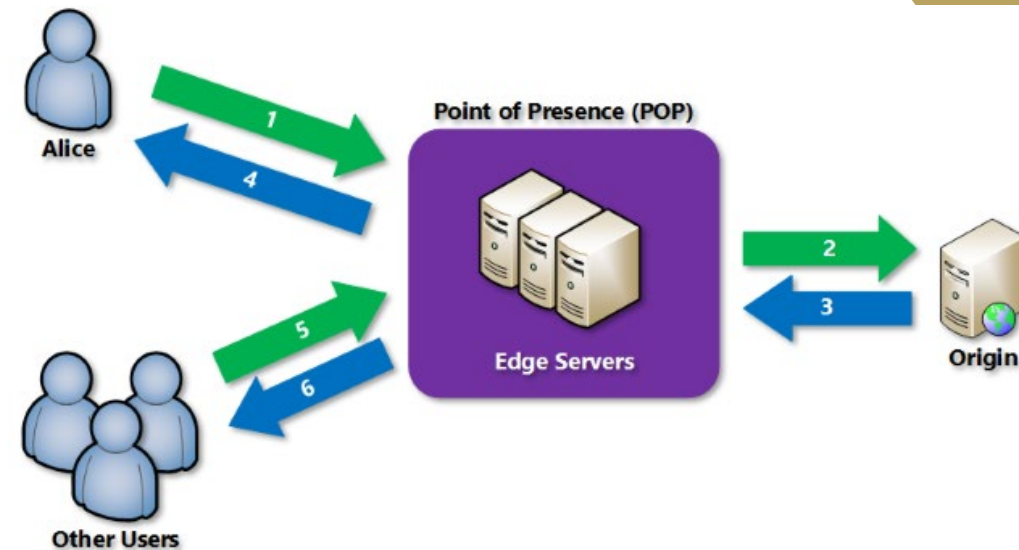
How it Works?

1. **A user (Alice) requests a file.** The DNS routes the request to the POP that is geographically closest to the user.
2. **If no edge servers in the POP have the file in their cache, the POP requests the file from the origin server.** The origin server can be an Azure Web App, Azure Cloud Service, Azure Storage account, or any publicly accessible web server.
3. The origin server **returns the file to an edge server in the POP.**



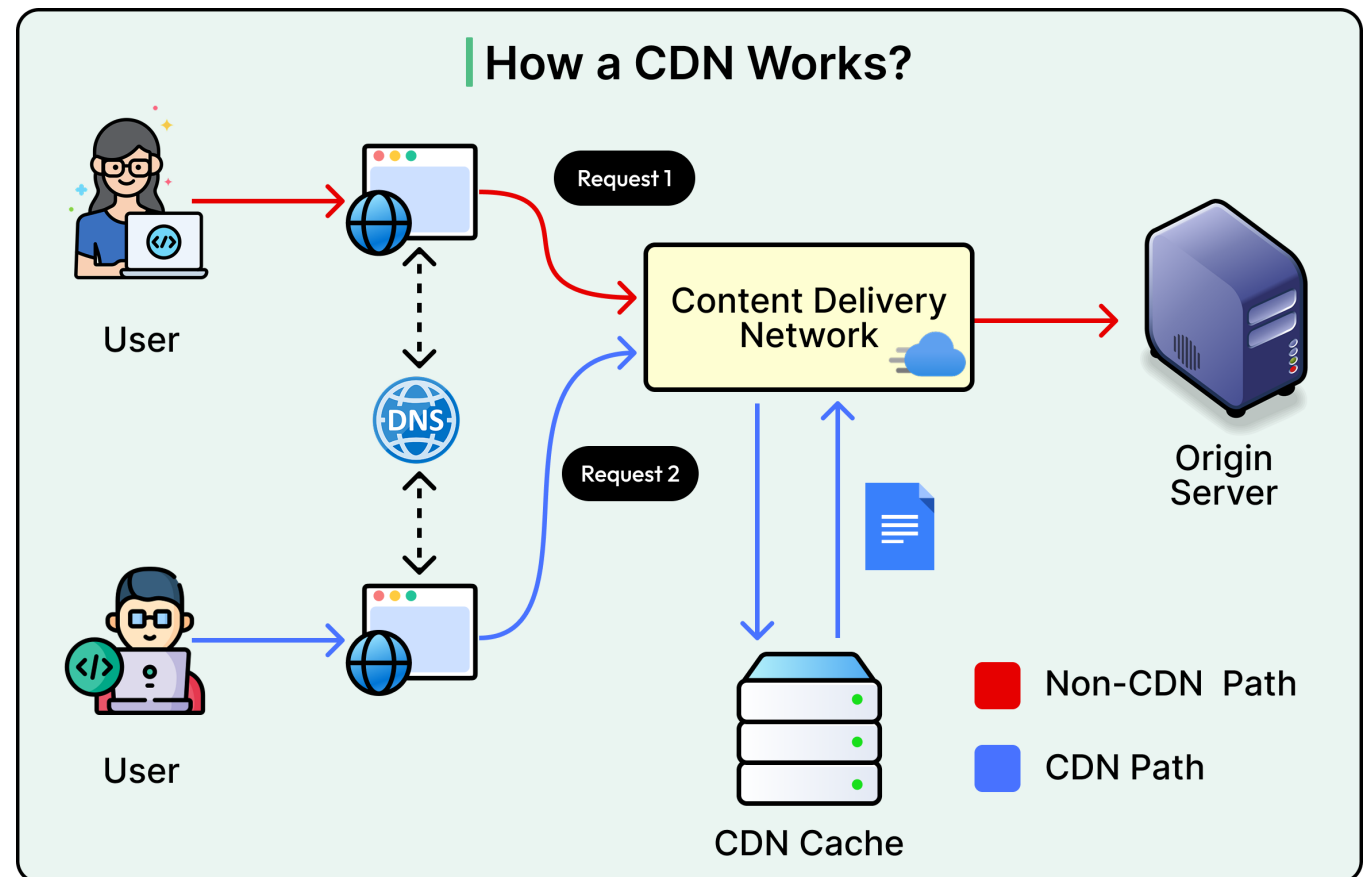
How it Works?

4. An edge server in the POP caches the file and returns the file to the original requestor (Alice).
5. Additional users can then request the same file by using the same URL that Alice used, and can also be directed to the same POP.
6. The POP edge server returns the file directly from the cache. **This process results in a faster, more responsive user experience.**



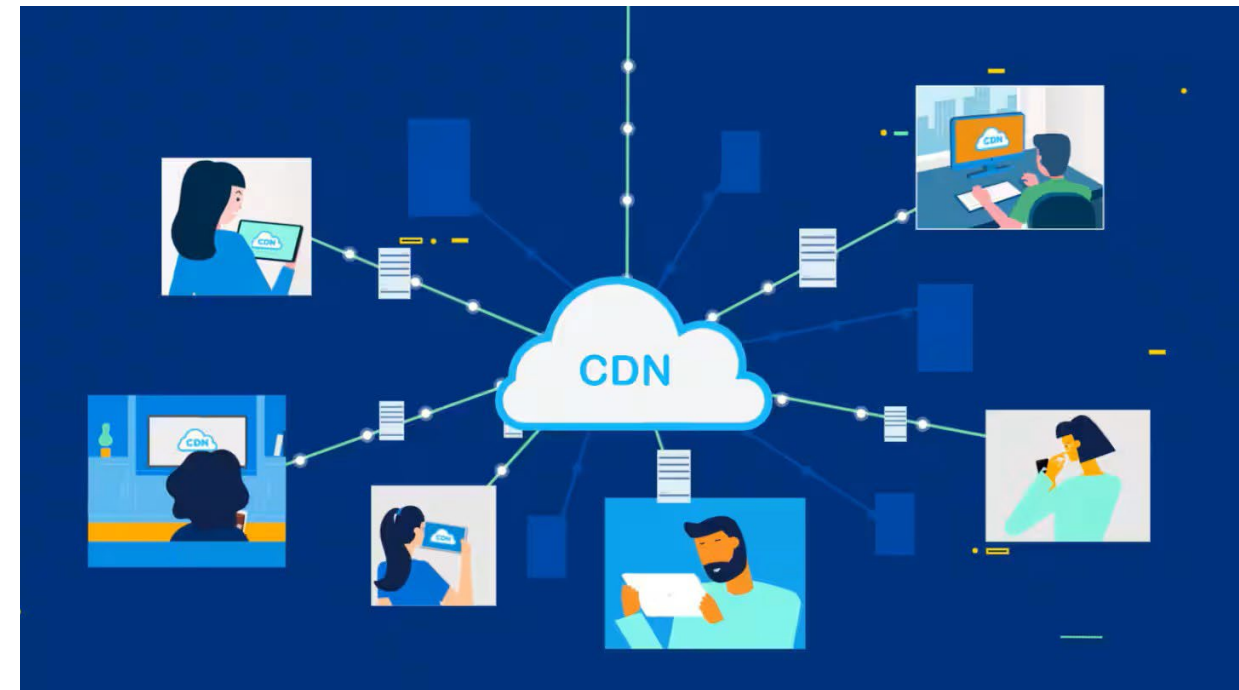
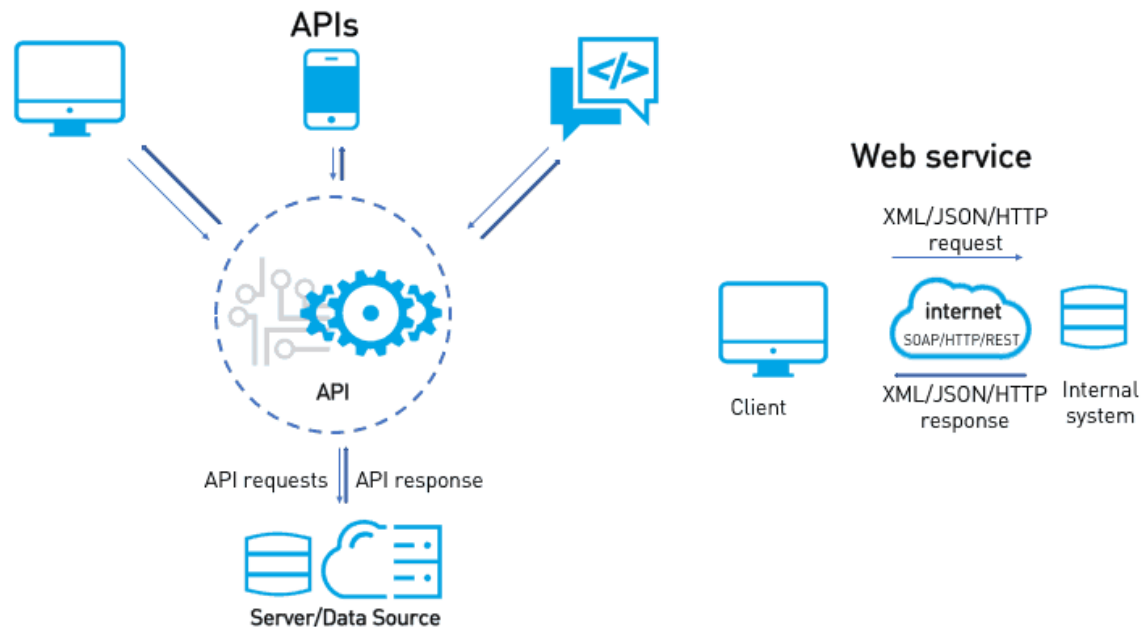
How a CDN is used?

- **Delivering static resources for client applications**, often from a website. These resources can be images, style sheets, documents, files, client-side scripts, HTML pages, HTML fragments, or any other content that the server does not need to modify for each request.



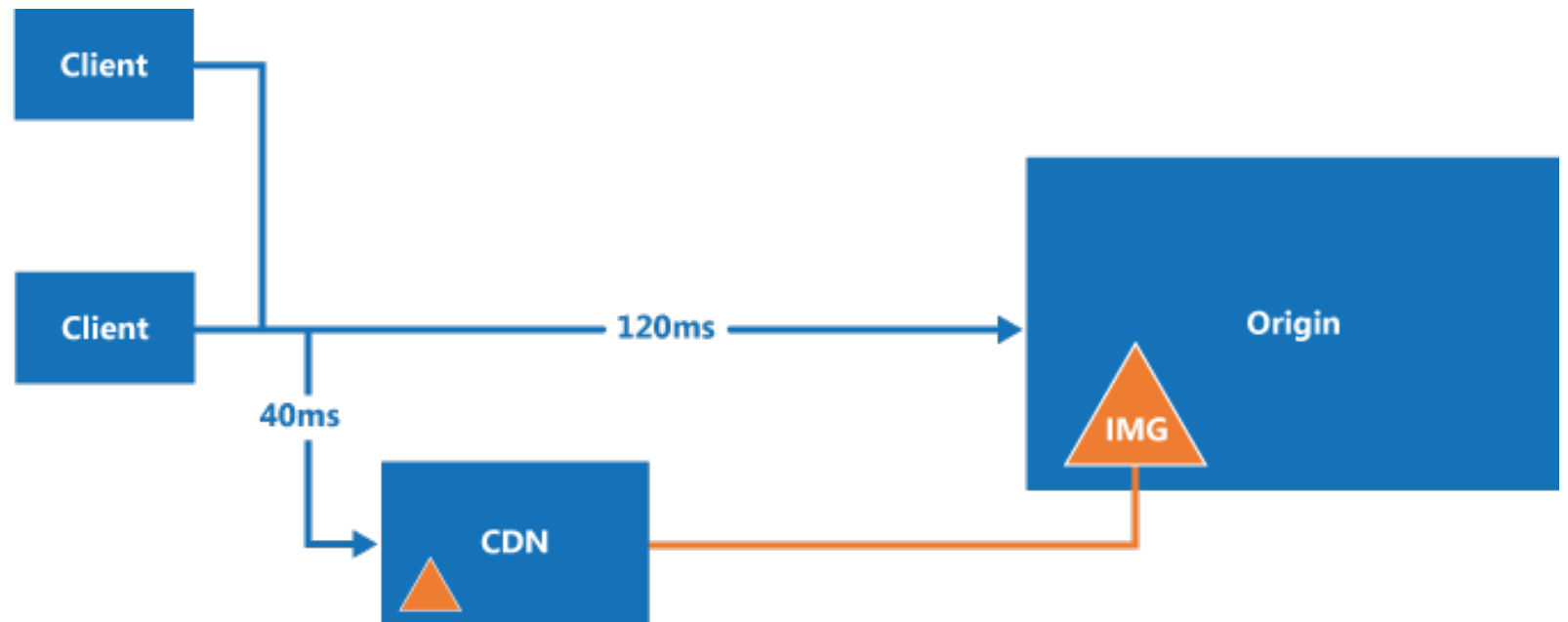
How a CDN is used?

Delivering public static and shared content to devices such as mobile phones and tablet computers. The application itself is a web service that offers an API to clients running on the various devices.



How and why a CDN is used?

Streaming video files to the client on demand. Video benefits from the low latency and reliable connectivity available from the globally located datacentres that offer CDN connections. Microsoft Azure Media Services (AMS) integrates with Azure CDN to deliver content directly to the CDN for further distribution.



Why a CDN is used?

Generally improving the experience for users, especially those located far from the datacentre hosting the application.

Supporting IoT (Internet of Things) solutions. The huge numbers of devices and appliances involved in an IoT solution could easily overwhelm an application if it had to distribute firmware updates directly to each device.

Coping with peaks and surges in demand without requiring the application to scale, avoiding the consequent increase in running costs. For example, when an update to an operating system is released for a hardware device such as a specific model of router, or for a consumer device such as a smart TV, there will be a huge peak in demand as it is downloaded by millions of users and devices over a short period.

Challenges with CDN

Deployment.

- Decide the origin from which the CDN fetches the content, and whether you need to deploy the content in more than one storage system.

Versioning and cache-control.

- Consider how you will update static content and deploy new versions. Understand how the CDN performs caching and time-to-live (TTL).

Testing.

- It can be difficult to perform local testing of your CDN settings when developing and testing an application locally or in a staging environment.

Search engine optimization (SEO).

- Content such as images and documents are served from a different domain when you use the CDN. This can have an effect on SEO for this content.

Content security.

- Not all CDNs offer any form of access control for the content. Some CDN services, including Azure CDN, support token-based authentication to protect CDN content.

Client security.

- Clients might connect from an environment that does not allow access to resources on the CDN.

Resilience.

- The CDN is a potential single point of failure for an application.

The Microsoft WAN

Microsoft believes that they operate the second largest private global dark fiber network in the world:

- 100,000 miles of fiber cable
- 54 Azure regions, with multiple data centers each (some still in construction, 2024)
- 130+ edge sites

Once you get a packet onto this WAN, you have an extremely low latency connection between any two points. For example, if a client connects to Azure in California, they have a low latency connection all the way to Microsoft services in The Netherlands – the speed cannot be matched on the public Internet where there would be many hops and much higher latency.



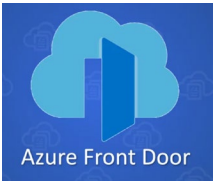
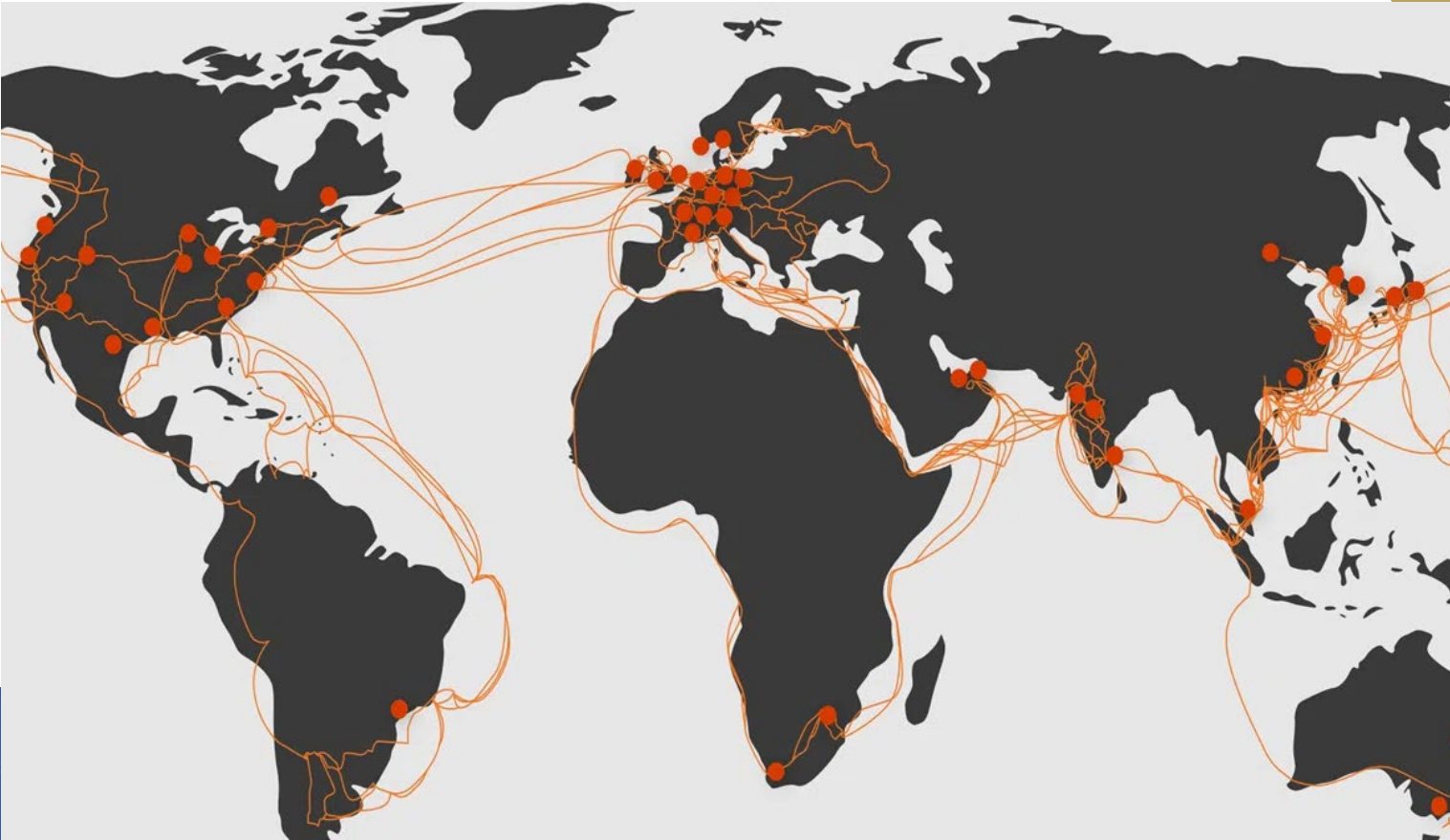
The Microsoft WAN

What if we could enable the client to enter the Microsoft WAN closer to their location, and connect to the Azure-hosted service across that WAN?

Front Door

Microsoft developed Front Door 5 years ago to enhance the performance of interactive services such as Office 365 and Bing. Since then, this globally deployed service has been battle tested by millions, if not billions, of users.

Front Door is not new – it’s newly available – making it an unusual “new” cloud service because it is already mature.

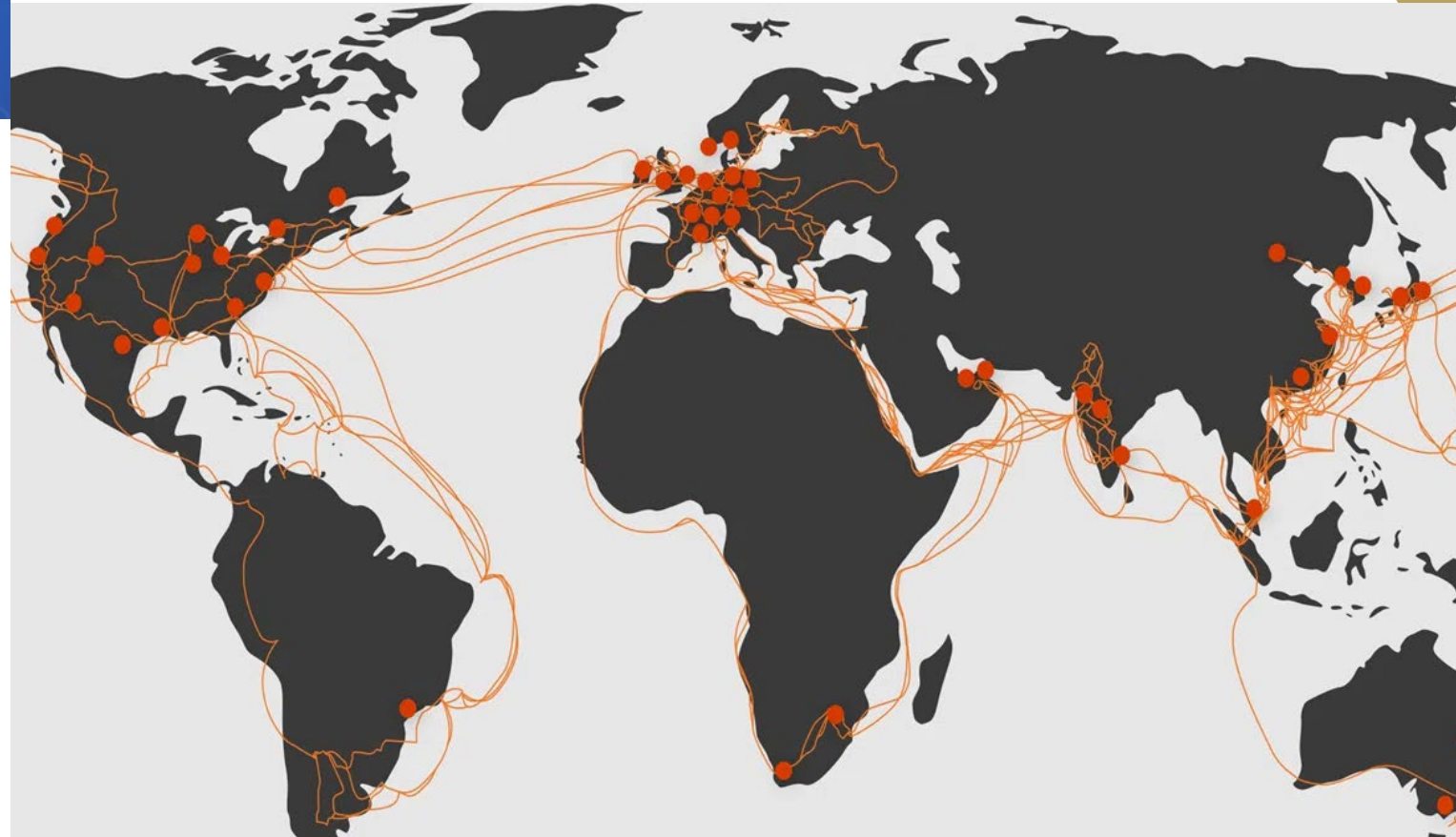




Front Door

Front Door is an entry point into the Microsoft WAN that is deployed in edge sites around the world.

When you connect to a service that Front Door is enhancing, you enter the Microsoft WAN through the closest (AnyCast) edge site and, from there, you connect to the closest available (probe tested) instance or replica of the service via the Microsoft WAN.



Azure Front Door

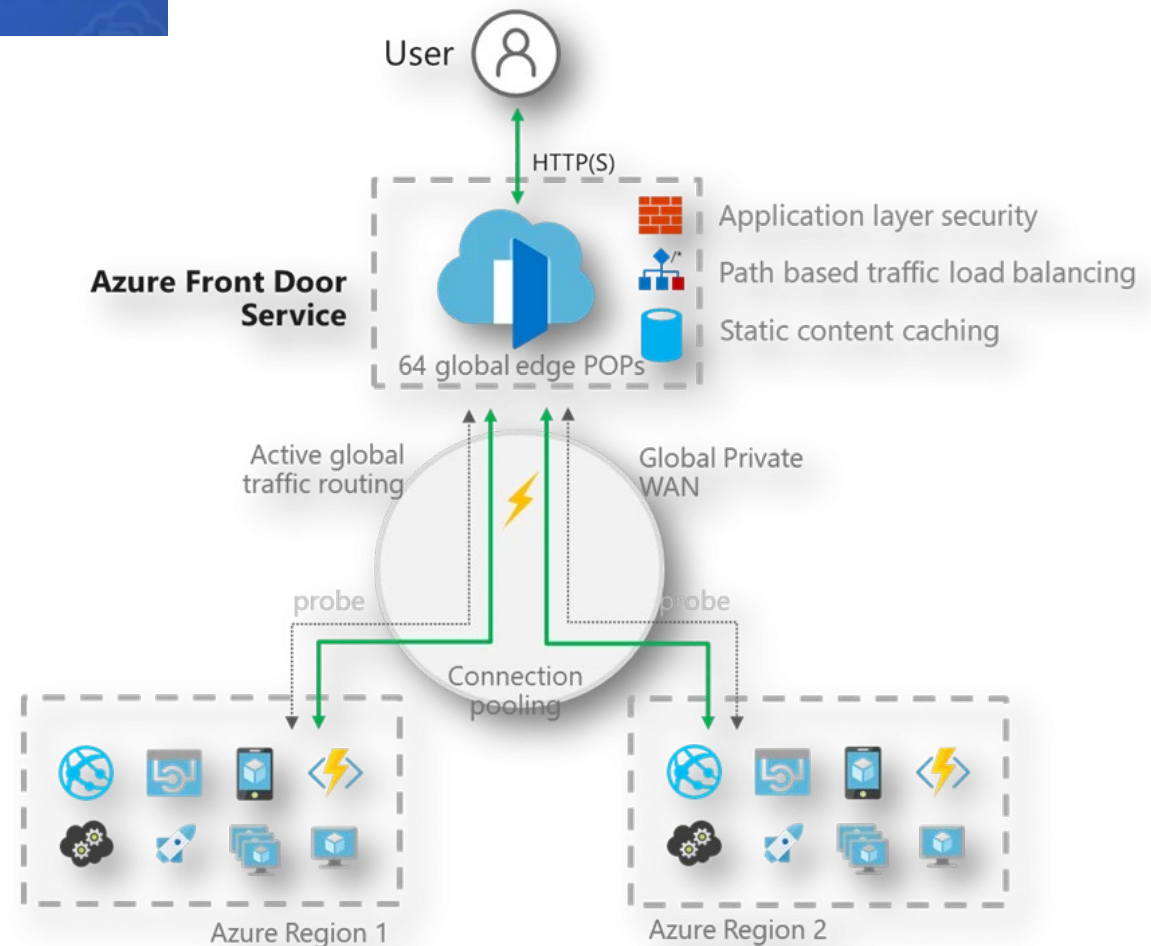


<https://youtu.be/-4FQYxV9mAE>

Azure Front Door



“Azure Front Door uses a TCP-based anycast protocol process as well as Microsoft’s global network, this further ensures higher availability and reliability - all while maintaining the performance boost” (Microsoft, 2018).

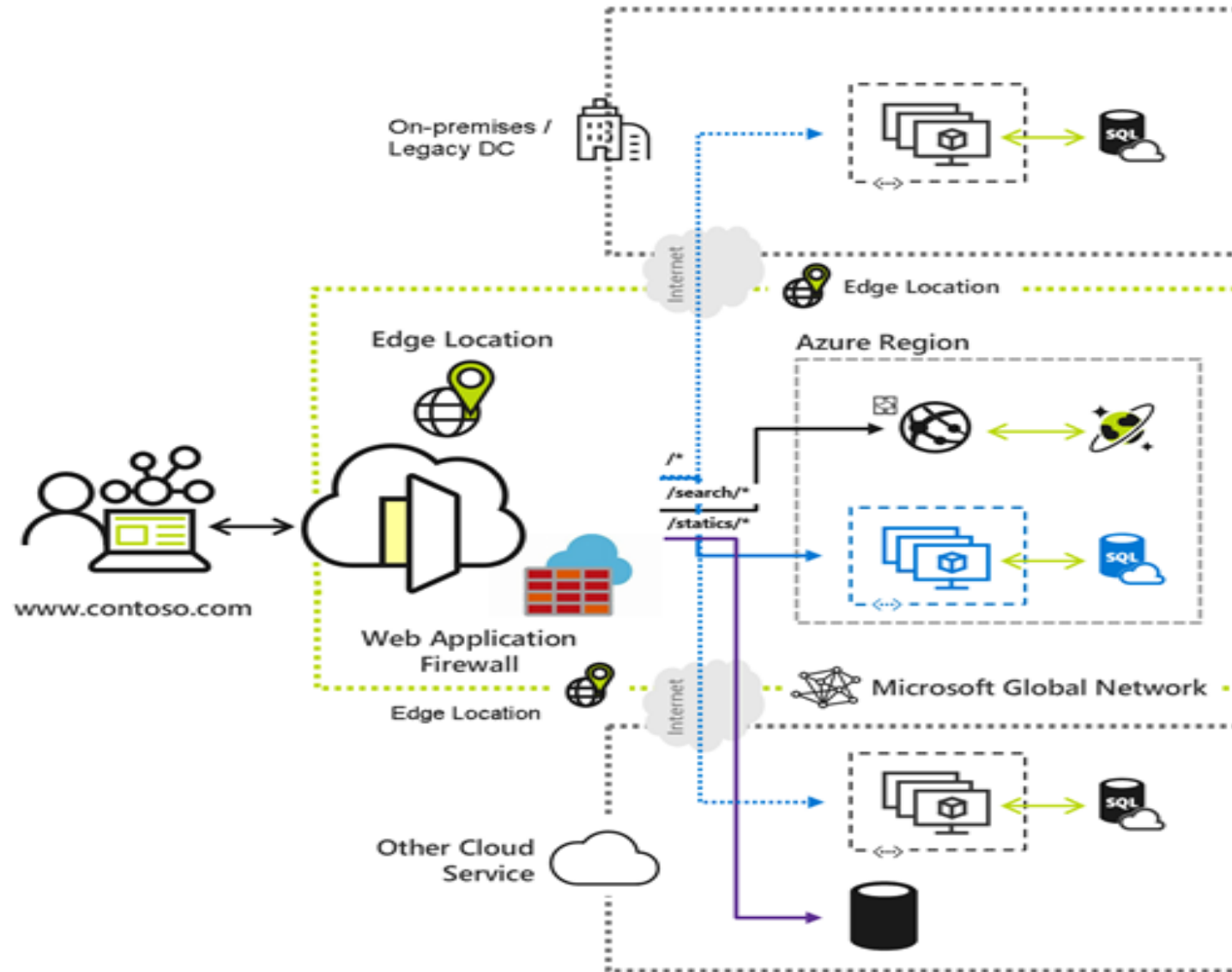


Azure Front Door



- Whether you're delivering **content and files** or **building global apps and APIs**, **Azure Front Door** can help you deliver higher availability, lower latency, greater scale, and more secure experiences to your users wherever they are.
- **Azure Front Door** is Microsoft's **modern cloud Content Delivery Network (CDN)** that provides fast, reliable, and secure access between your users and your applications' static and dynamic web content across the globe.
- **Azure Front Door** delivers your content using Microsoft's global edge network with hundreds of global and local **points of presence (PoPs)** distributed around the world close to both your enterprise and consumer end users.

**Ulster
University**



Why use Azure Front Door?

Azure Front Door enables internet-facing application to:

- **Build and operate modern internet-first architectures** that have dynamic, high-quality digital experiences with highly automated, secure, and reliable platforms.
- **Accelerate and deliver your app and content globally** at scale to your users wherever they're creating opportunities for you to compete, weather change, and quickly adapt to new demand and markets.
- **Intelligently secure your digital estate** against known and new threats with intelligent security that embrace a **Zero Trust** framework.



Scalability



Scalability

Scalability is **the ability of a system to handle the increased load**. Services covered by Azure Auto-scale can scale automatically to match demand to accommodate the workload.

There are three main types of scalability in cloud computing: horizontal scalability, vertical scalability, and hybrid scalability. Each type offers unique benefits and is designed to meet different business needs.

What Are The Types Of Scaling In Cloud Computing?

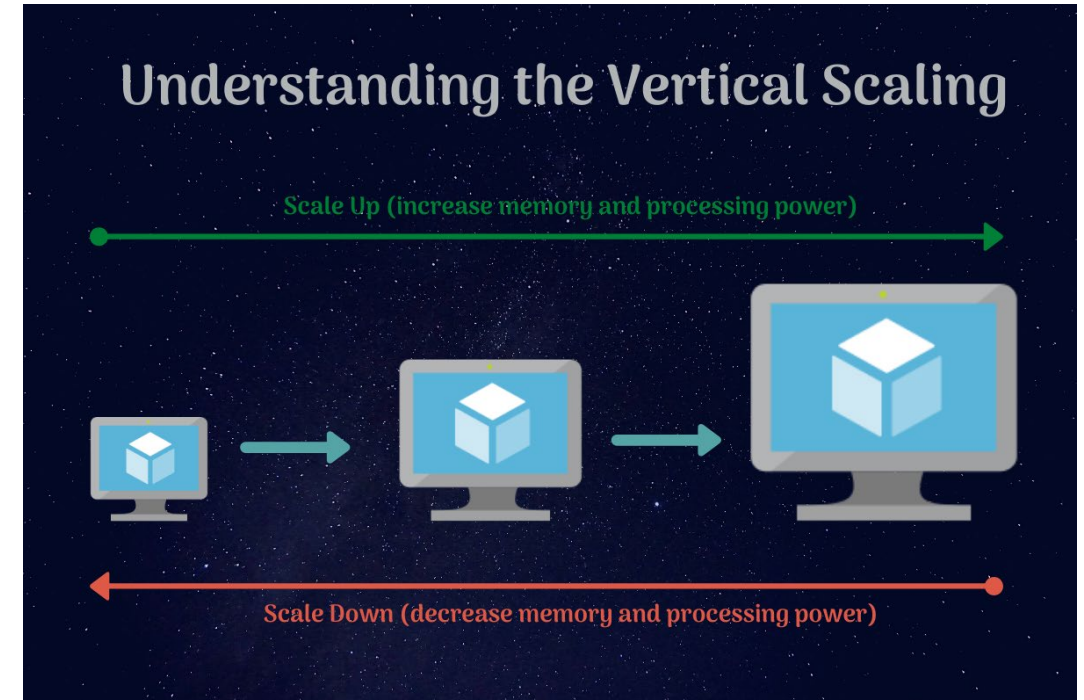
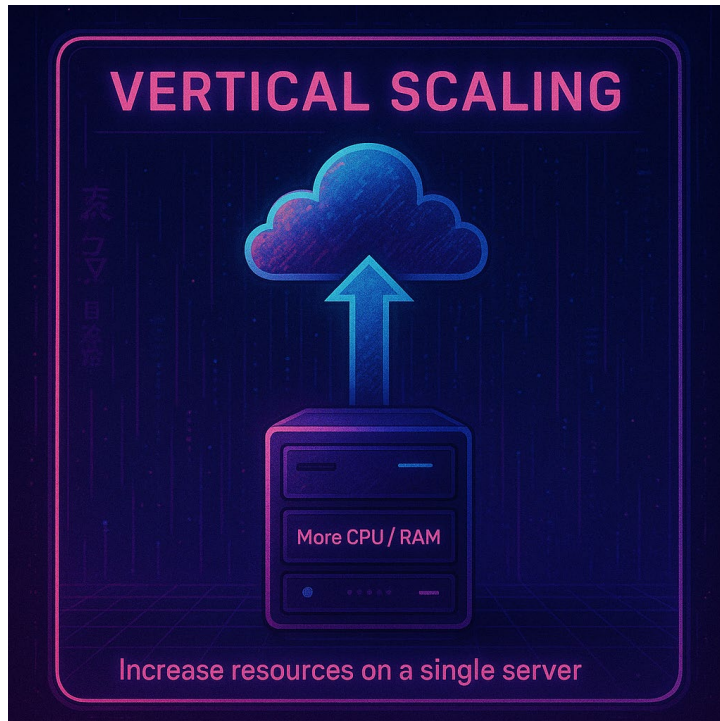


Scalability

Scaling Up

- Vertical scaling or scale-up is applicable to almost all Azure services.

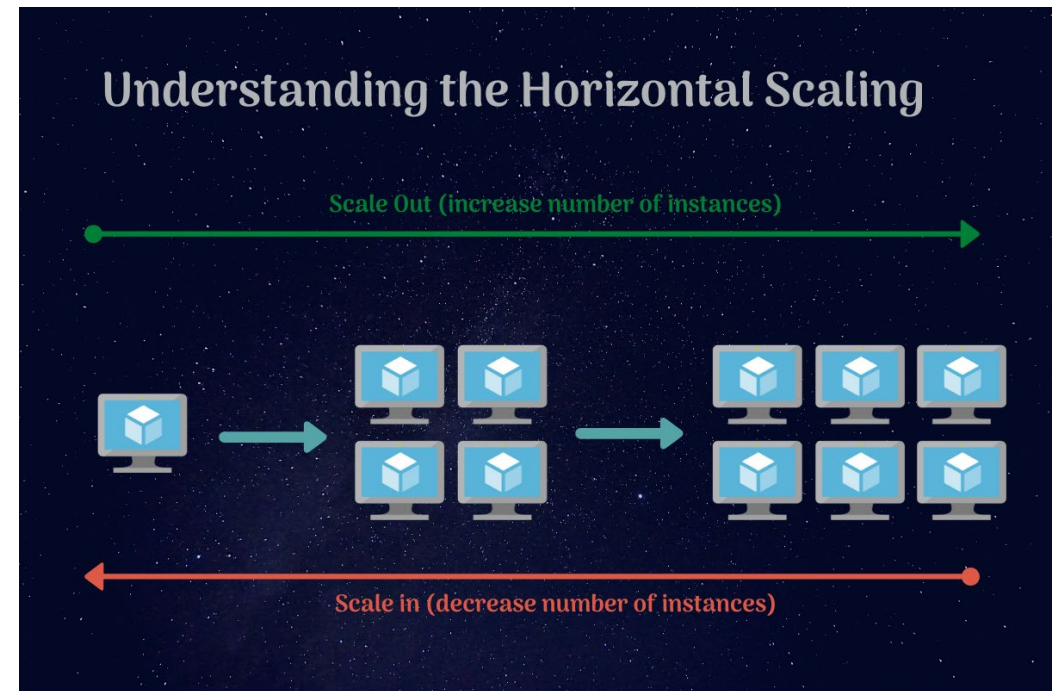
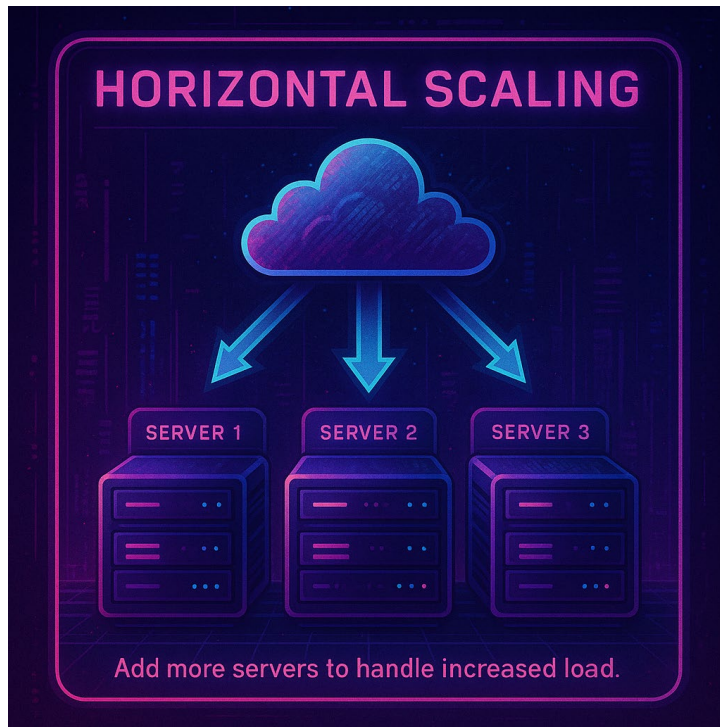
For example, applying scale-up to a virtual machine, **means changing the size of the virtual machine.**



Scalability

Scaling Out

- Vertical scaling can be done by changing the pricing tiers in Azure. But there is limit on how many times are you going to scale up.
- Eventually it will hit the limit because there is a limit of memory or processor that a computer can support. That's why horizontal scaling is helpful.



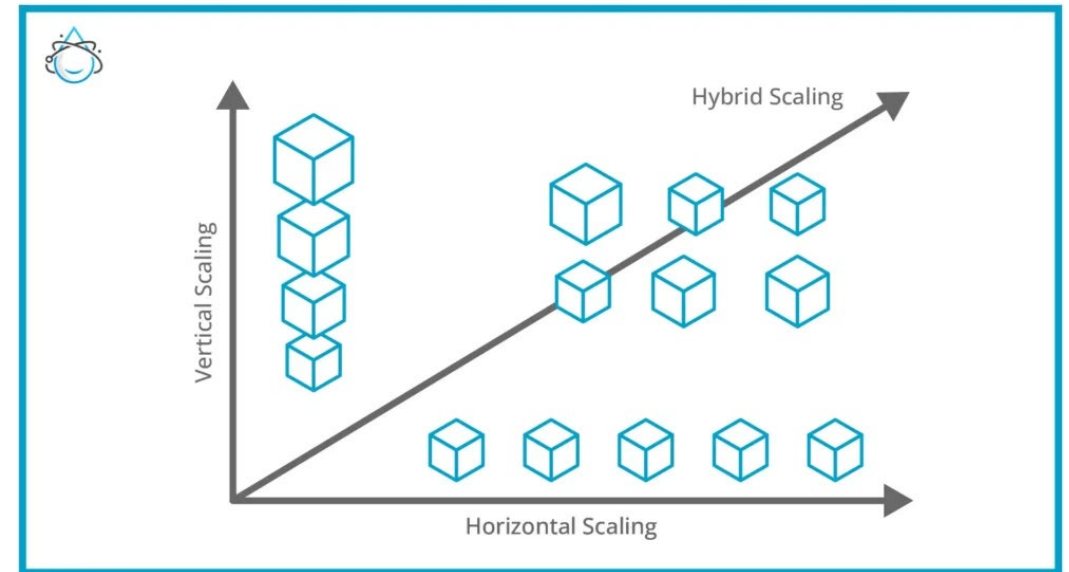
Scalability

Hybrid Scaling

Hybrid scalability, also known as diagonal scaling, combines both horizontal and vertical scalability to provide a flexible and scalable cloud environment.

This type of scalability allows you to add more instances or resources as needed while also optimizing your existing resources to achieve maximum efficiency.

Hybrid scalability is often used for complex applications that require a combination of processing power, storage, and bandwidth.



Scalability

Benefits

Cloud scalability isn't just about handling growth—it's about doing it efficiently. When your infrastructure scales the right way, you avoid overprovisioning, reduce downtime risks, and support faster product iteration. The benefits of scalability in cloud computing include:

What Are The Benefits Of Cloud Scalability?



Cost Savings



Improved
Performance



Flexibility and
Agility



Improved Availability
and Reliability



Enhanced Security

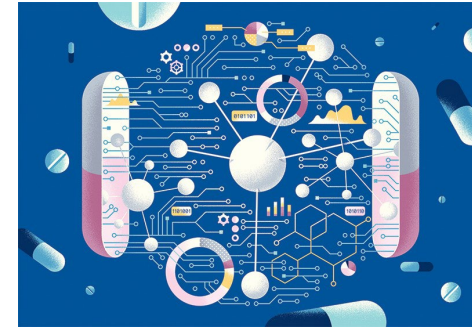
Scalability

Do we really need this?

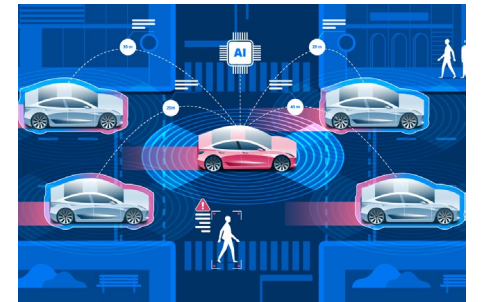
Modern IT infrastructure runs on distributed systems, containers, and data-heavy workloads. Here's how cloud scalability plays a critical role in modern IT infrastructure:

Enables AI/ML and data-intensive workloads

Modern use cases—like model training or real-time analytics—require infrastructure that can scale rapidly with compute and memory demands.



Drug discovery



Smart vehicles network

Scalability

Do we really need this?

Facilitates global, always-on services

Cloud scalability ensures consistent performance across regions and time zones, supporting 24/7 availability for global applications.



TARGET RAPID RESPONSE 

24/7 EMERGENCY SERVICES

Are you experiencing issues with being locked out, clogged drains, or the need for window repair and glass replacement? Look no further as we are here to provide solutions to all your problems!

-  Locksmith
-  Drainage
-  Board Up

CONTACT US

Visit Our Website  www.targetrapidresponse.co.uk Contact Us  0800 121 4721

